Покадровый сценарий мастер-класса

Toolkit программиста, или к чему обратиться, чтобы закрыть проект в сроки

Версия для согласования содержания и базовой вёрстки. 29 экранов, включая перерыв. 90 минут занятия + ориентировочно 10 минут перерыва.

Условия проведения

- Один ведущий, 20 школьников 10-11 классов. Четыре команды примерно по пять человек.
- Общий экран; голосование руками. Ведущий фиксирует итоговый выбор. Телефоны, компьютеры участников и интернет для сценария не требуются.
- Продукт разрабатывать не нужно. Короткий пример Python разбирается устно на экране.
- Сквозной учебный кейс: сервис записи на школьный фестиваль. За неделю нужно подготовить первую демонстрацию с тестовыми данными.

Как читать документ

- Каждая следующая страница сценария соответствует одному экрану будущего сайта. Пункты «На экране» - содержание для участников; остальные разделы - заметки ведущего.
- «Речь ведущего» содержит опорные тезисы, а не текст для чтения. «Раскрытие и переход» показывается после обсуждения или произносится устно.
- Время указано от начала учебной части. После перерыва счётчик учебного времени продолжается с 45:00; по обычным часам добавится длительность перерыва.

Что считается результатом

- Участник связывает проблему со способом решения, а не просто запоминает названия сервисов.
- У группы собран общий набор из пяти разделов: требования, организация работы, совместный код, ИИ, проверка и выпуск.
- В финале команды обосновывают порядок действий в ситуации с ограниченным временем.

Карта занятия и базовая вёрстка

Блок	Экраны	Учебное время
Вступление и кейс	01-02	00:00-05:00
Требования и MVP	03-05	05:00-15:00
Agile и Scrum	06-09	15:00-30:00
Общие материалы, Git, review	10-13	30:00-45:00
Перерыв	14	Около 10 минут отдельно
Нейросети	15-19	45:00-60:00
Тесты, CI/CD, восстановление	20-24	60:00-75:00
Финальное испытание	25-27	75:00-85:00
Закрепление	28-29	85:00-90:00

Структура одного экрана

- Верхняя строка: раздел и номер экрана. Крупный заголовок. Центральная область: условие, пример, доска или код.
- Вопрос и варианты выбора находятся ниже условия. На одном шаге видны только материалы, нужные для текущего обсуждения.
- После ответа ведущий раскрывает вывод; он заменяет часть содержимого либо открывается следующим состоянием того же экрана.
- Набор команды можно кратко показывать на итоговых экранах разделов. Не держать все определения и подсказки на экране постоянно.
- Заметки ведущего не выводятся участникам по умолчанию. Будущий способ их просмотра определим при проектировании сайта.

Правило голосований

- До раскрытия ответов: 30-60 секунд обсуждения, поднятие рук, одна защита решения. При ничьей - короткий аргумент от сторон и повторное голосование; при повторной ничьей ведущий выбирает учебный вариант.
- В первом МК выбор влияет на обсуждение, но после разбора всех вариантов группа возвращается к общей линии. Полноценное ветвление истории оставлено второму МК.

Экран 01. Сегодня вы принимаете решения

2 мин | 00:00-02:00

Базовая вёрстка: Заголовок, задача занятия, три правила.

На экране

- Toolkit программиста, или к чему обратиться, чтобы закрыть проект в сроки
- Наша цель: собрать набор способов работы, который поможет довести проект до демонстрации.
- Обсуждаем в командах. Голосуем руками. Объясняем свой выбор.

Речь ведущего

- Назвать себя и обозначить формат: решения будут принимать участники, ты помогаешь разбирать последствия.
- Уточнить: писать полноценное приложение сегодня не нужно; важнее понять, как работает команда.
- Разделить зал на четыре команды примерно по пять человек. Представитель может меняться.

Вопросы и действия участников

- Спросить: «Что чаще мешает закончить проект: сложный код, непонятная задача или работа команды?»
- Принять 2-3 коротких ответа без попытки сразу определить победителя.

Раскрытие и переход

- Все три причины возможны. Будем искать средство под конкретную проблему.
- Переход: «Представим, что именно вашей команде поручили школьный сервис».

Управление временем: Если команды уже рассажены, не тратить время на их названия и распределение постоянных ролей.

Экран 02. Одна неделя до школьного фестиваля

3 мин | 02:00-05:00

Базовая вёрстка: Условие слева, запрос заказчика справа.

На экране

- Проект: сервис записи на школьные мероприятия.
- Пользователь выбирает мероприятие и записывается. Количество мест ограничено.
- Организатор видит список участников. Первая демонстрация через неделю.
- Пожелания: профили, рекомендации, уведомления и красивый дизайн.

Речь ведущего

- Объяснить, кто пользуется сервисом и зачем: школьнику не нужно искать организатора в коридоре, организатору - вручную собирать списки.
- Различить демонстрацию и полноценный запуск: для показа можно использовать тестовые данные, но не выдавать прототип за готовую систему.
- В условном проекте работает небольшая команда; реальные участники МК обсуждают её решения.

Вопросы и действия участников

- Попросить команды за 30 секунд назвать первый шаг.
- Собрать три ответа: начать код, уточнить задачу, выбрать технологии.

Раскрытие и переход

- Сначала нужно понять, какой результат действительно нужен. Технологии выбираются с учётом этой задачи.
- Переход: «Что в нашей задаче пока непонятно?»

Важно: Не обещать выпуск реального продукта за неделю. Это условие учебного кейса, а не оценка трудозатрат.

Экран 03. Сначала выясняем, что значит «работает»

3 мин | 05:00-08:00

Базовая вёрстка: Три вопроса заказчику; ответы появляются после предложений зала.

На экране

- Кто будет пользоваться сервисом?
- Что пользователь должен суметь сделать?
- Как мы убедимся, что это получилось?

Речь ведущего

- Объяснить требования через наблюдаемое поведение, а не через названия технологий.
- Попросить представить себя заказчиком: фраза «сделайте удобно» не объясняет, какое поведение ожидается.
- Примеры уточнений: запись на одно или несколько мероприятий; повторная запись; отмена; кто видит список участников.

Вопросы и действия участников

- Команды предлагают по одному вопросу заказчику.
- Для продолжения зафиксировать: повторная запись запрещена; отмену и сложные ограничения можно отложить; данные пока тестовые.

Раскрытие и переход

- Рабочие требования: увидеть список, записаться при наличии мест, не записаться повторно, показать организатору список.
- Переход: «Теперь выберем, что успеем показать в первой версии».

Управление временем: Взять два вопроса из зала, остальные проговорить самому.

Экран 04. Выбираем минимальную полезную версию

4 мин | 08:00-12:00

Базовая вёрстка: Шесть функций; ниже варианты A/B/C. После выбора - разбор всех вариантов.

На экране

- A. Список мероприятий + запись + лимит мест + список участников.
- B. Профили + красивые анимации + рекомендации.
- C. Все функции сразу, даже если часть будет работать нестабильно.
- Голосуем за основу первой демонстрации.

Речь ведущего

- Дать командам 45 секунд на обсуждение. Попросить назвать не только выбранные функции, но и решаемую задачу.
- Ввести MVP: минимальная версия, с помощью которой можно проверить основную ценность продукта.
- Отметить: минимум определяется целью; для проверки дизайна мог бы подойти прототип, но наш кейс проверяет именно запись.

Вопросы и действия участников

- Голосование руками A/B/C; один голос на человека.
- Попросить одну команду объяснить выбор и другую назвать его ограничение.

Раскрытие и переход

- A: проверяет основной путь, но дополнительные пожелания откладываются.
- B: может проверить внешний вид и интерес, но не работу записи. C: широкий объём увеличивает риск не довести основной путь до готовности.
- Для общей учебной линии далее рассматриваем A; выбор зала сохраняем для обсуждения, но не меняем весь сценарий.

Важно: Первый МК - разбор решений. После любого голосования возвращаемся к общей учебной линии, а не создаём отдельную сюжетную ветку.

Экран 05. Устанавливаем критерии готовности

3 мин | 12:00-15:00

Базовая вёрстка: Требование и три примера; затем первая строка набора команды.

На экране

- При наличии места новая запись создаётся.
- При заполненном мероприятии запись отклоняется.
- Повторная запись того же участника отклоняется.
- В набор: понятные требования + MVP + критерии приёмки.

Речь ведущего

- Показать отличие пожелания «запись работает» от проверяемого условия.
- Критерии приёмки относятся к конкретному требованию; общие требования качества обсудим позже.
- Предложить один пограничный пример: осталось ровно одно место.

Вопросы и действия участников

- Спросить: «Если кнопка красиво нажимается, но запись не появляется в списке, задача готова?»
- Попросить сформулировать одно условие готовности без слова «нормально».

Раскрытие и переход

- Готовность определяется согласованным результатом, а не объёмом написанного кода.
- Переход: «Как организовать неделю так, чтобы узнать о проблемах до последнего вечера?»

Экран 06. Agile: проверяем направление по пути

3 мин | 15:00-18:00

Базовая вёрстка: Два плана рядом; затем цикл из четырёх действий.

На экране

- План А: неделю делать всё, показать в последний день.
- План В: сделать небольшой работающий шаг, показать, получить обратную связь, уточнить следующий шаг.
- Сделали → проверили → получили обратную связь → скорректировали.

Речь ведущего

- Назвать Agile подходом, основанным на ценностях и принципах гибкой разработки, а не приложением для управления задачами.
- Подчеркнуть работающий результат, сотрудничество и возможность учиться на обратной связи.
- Объяснить: план остаётся нужен; меняется готовность пересматривать его по новым данным.

Вопросы и действия участников

- Спросить: «Когда дешевле узнать, что школьники пользуются только телефоном: после первого экрана или после всей разработки?»
- Получить один аргумент в пользу короткой проверки.

Раскрытие и переход

- Короткие циклы помогают раньше обнаружить неверные предположения; сами по себе они не гарантируют успех.
- Переход: «Scrum - один из способов организовать такую работу».

Важно: Не противопоставлять Agile любому планированию и документации. Вопрос в полезности и адаптации, а не в их полном отказе.

Экран 07. Scrum: короткий цикл с понятной целью

4 мин | 18:00-22:00

Базовая вёрстка: Цель цикла; роли ответственности и упрощённая последовательность.

На экране

- Цель ближайшего цикла: показать работающую запись на одно мероприятие.
- Product Owner: ценность и порядок задач. Developers: создание результата. Scrum Master: помогает применять Scrum.
- Планирование → работа и ежедневная сверка → обзор результата → улучшение процесса.

Речь ведущего

- Объяснить спринт как ограниченный по времени цикл. В Scrum он длится не более месяца; наш разговор - лишь учебная иллюстрация.
- Назвать бэклог упорядоченным списком нужной работы. Цель спринта объясняет, ради чего выбрана её часть.
- Daily Scrum - короткое ежедневное планирование Developers к цели спринта, не отчёт начальнику.
- Обзор: обсудить продукт с заинтересованными людьми. Ретроспектива: улучшить способ работы.

Вопросы и действия участников

- Назначить на один вопрос роли: заказчик хочет больше функций; разработчик сообщает, что базовая запись ещё не проверена.
- Спросить зал: «Какую цель цикла выбираем?»

Раскрытие и переход

- Зафиксировать одну понятную цель вместо обещания «сделать как можно больше».
- Переход: «Теперь нужно увидеть, какая работа ведёт к этой цели».

Важно: Это знакомство с элементами Scrum, а не полноценная симуляция всего фреймворка. Scrum Master не равен начальнику команды.

Экран 08. Доска задач: видим работу и препятствия

4 мин | 22:00-26:00

Базовая вёрстка: Четыре колонки и 4-5 коротких карточек.

На экране

- Нужно сделать | В работе | На проверке | Готово
- Карточки: список мероприятий; запись; проверка лимита; список участников.
- У каждой задачи - понятный результат и ответственный за следующий шаг.

Речь ведущего

- Объяснить движение одной карточки: написанный код ещё может ждать проверки.
- Показать препятствие: проверяющий не знает ожидаемое поведение, потому что критерии не записаны.
- Не вводить новые сервисы как обязательные: для доски подходят бумага, GitHub Projects, Trello и другие инструменты.

Вопросы и действия участников

- Вызвать представителя и предложить разместить карточки по условию: список показали, запись написали, лимит ещё не проверяли.
- Спросить: «Почему нельзя поставить всё в колонку “В работе”?»

Раскрытие и переход

- Слишком много начатого затрудняет завершение; полезно договориться об ограничении одновременной работы.
- Доска делает работу видимой. Одна доска сама по себе не означает внедрение Scrum или всей системы Kanban.
- Переход: «А если в середине работы появляется новое пожелание?»

Управление временем: Карточки перемещаешь ты по предложениям зала. Выход участника к экрану необязателен.

Экран 09. Заказчик изменил требования

4 мин | 26:00-30:00

Базовая вёрстка: Новость; три варианта; после выбора - последствия.

На экране

- Новость: сервис будут открывать с телефона. Заодно заказчик просит уведомления.
- A. Добавить всё, сохранив прежние обещания.
- B. Уточнить важность, сохранить цель записи, согласовать обмен объёма и приоритетов.
- C. Не обсуждать никакие изменения до конца недели.

Речь ведущего

- Разделить новое обязательное условие использования и дополнительную функцию.
- Адаптация под телефон может быть нужна основному пути. Уведомления можно оценить отдельно.
- Уточнить: в Scrum нельзя допускать изменений, ставящих под угрозу цель спринта; объём может уточняться по мере понимания задачи.

Вопросы и действия участников

- 45 секунд обсуждения, голосование, одна короткая защита.
- Спросить: «Что вы отложите, если проверка на телефоне займёт больше времени?»

Раскрытие и переход

- A: растёт нагрузка без пересмотра ожиданий. B: появляется явная договорённость о приоритетах. C: полезная информация может быть потеряна.
- B набор: обратная связь + цель короткого цикла + видимые задачи.
- Переход: «Мы договорились о работе. Как не потерять договорённости и код?»

Экран 10. Общие знания и общий код

3 мин | 30:00-33:00

Базовая вёрстка: Две области: договорённости и исходный код.

На экране

- Где описано, что делаем: требования, решения, инструкция запуска.
- Где меняем код: репозиторий, история изменений, предложения на объединение.
- Один понятный источник актуальной информации для каждого типа материалов.

Речь ведущего

- Начать с бытовой проблемы: важное решение есть только в личной переписке одного человека.
- Для знаний подходят README, wiki, общий документ; для отслеживания задач - доска или issue tracker.
- Файловая папка удобна для раздачи материалов, но не заменяет возможности контроля версий при совместном изменении кода.
- Документация может храниться вместе с кодом; разделение на экране смысловое, а не требование двух разных сервисов.

Вопросы и действия участников

- Попросить распределить три объекта: правило запрета повторной записи; исходный файл; задача об ошибке.

Раскрытие и переход

- Ответы: требования/документация; репозиторий; задача с шагами воспроизведения.
- Переход: «Посмотрим, чем история кода лучше папки final_final».

Экран 11. Git: история изменений вместо final_final

4 мин | 33:00-37:00

Базовая вёрстка: История из трёх изменений; определения показываются рядом.

На экране

- Коммит: исправили проверку мест.
- Коммит: добавили список участников.
- Коммит: улучшили сообщение об отказе.
- Git - контроль версий. GitHub/GitLab - платформы совместной работы.

Речь ведущего

- Коммит фиксирует состояние проекта; полезное сообщение объясняет сделанное изменение.
- Репозиторий хранит проект и его историю. У каждого разработчика может быть своя локальная копия.
- Git помогает сравнивать версии, понимать изменения и возвращаться к прежним состояниям.
- Сохранить файл, сделать коммит и отправить изменения в общий репозиторий - разные действия. Термины push/pull можно назвать без команд.

Вопросы и действия участников

- Спросить: «Ноутбук сломался до отправки локальных изменений. Обязательно ли они уже есть у коллег?»
- Спросить: «Что легче проверить: одно понятное изменение или сотню несвязанных?»

Раскрытие и переход

- Неотправленные изменения могут оставаться только на одном устройстве.
- Небольшие осмысленные изменения проще обсуждать и проверять.
- Переход: «А если два человека поменяли одно место?»

Экран 12. Два изменения в одной функции

4 мин | 37:00-41:00

Базовая вёрстка: Исходная версия и два варианта рядом; ответ скрыт до обсуждения.

На экране

- Исходно: проверяем, осталось ли место.
- Разработчик A: меняет условие на «занято меньше вместимости».
- Разработчик B: меняет то же условие и добавляет запрет повторной записи.
- Как сохранить оба нужных изменения?

Речь ведущего

- Объяснить ветку как отдельную линию изменений, а не копию всего проекта в новой папке.
- При объединении Git может автоматически совместить часть правок, но иногда требуется решение человека.
- Даже автоматическое объединение не доказывает, что общая логика правильная.

Вопросы и действия участников

- Представитель одной команды защищает лимит мест, другой - запрет повторной записи.
- Зал формулирует итоговое правило: свободное место И отсутствие прежней записи.

Раскрытие и переход

- Не выбирать вслепую «всё от A» или «всё от B». Понять намерение изменений и проверить совместный результат.
- Конфликт текста и ошибка поведения - разные проблемы.
- Переход: «Кто и как проверит предлагаемое изменение?»

Управление временем: Не показывать синтаксис конфликтных маркеров Git: для цели МК достаточно столкновения двух намерений.

Экран 13. Перед объединением: проверка изменений

4 мин | 41:00-45:00

Базовая вёрстка: Последовательность и три правила команды.

На экране

- Ветка → небольшое изменение → запрос на объединение → review и проверки → общая версия.
- В запросе: что меняем, зачем и как проверили.
- В набор: репозиторий + история + code review.

Речь ведущего

- Pull request / merge request - предложение объединить изменения; reviewer обсуждает код и задаёт вопросы.
- Проверять должен подходящий по компетенции участник. Название должности само по себе не гарантирует качество.
- Для учебного проекта не нужна тяжёлая бюрократия: важны понятные изменения и быстрая обратная связь.
- Правило проверок нужно настроить и соблюдать: наличие GitHub само по себе не запрещает опасные изменения.

Вопросы и действия участников

- Голосование: «Что делаем перед демонстрацией: сразу меняем общую версию, просим быструю проверку, переносим любые изменения?»
- Уточнить условие: изменение необходимо, до показа есть время на короткую проверку.

Раскрытие и переход

- При этом условии выбираем небольшую правку с review и проверкой критичного поведения.
- Кратко назвать три уже собранные части набора и объявить перерыв.

Важно: Review уменьшает риск, но не заменяет тестирование и не гарантирует отсутствие ошибок.

Экран 14. Перерыв

10 мин | После 45 минут; не входит в 90 минут занятия

Базовая вёрстка: Один заголовок и время возвращения.

На экране

- Перерыв - ориентировочно 10 минут.
- После перерыва: ИИ, проверки и выпуск изменений.

Речь ведущего

- Назвать реальное время возвращения по часам.
- Оставить экран перерыва, не открывать ответы следующего блока.
- Если нужно, уточнить у участников, какие термины остались непонятными.

Вопросы и действия участников

- Обязательных заданий нет.

Раскрытие и переход

- После возвращения перейти к экрану 15.

Управление временем: Длительность перерыва согласовать с фактическим расписанием и состоянием группы.

Экран 15. Какую работу поручим ИИ?

2 мин | 45:00-47:00

Базовая вёрстка: Три примера задач и вопрос.

На экране

- Объяснить незнакомую функцию.
- Предложить черновик проверки записи.
- Подобрать примеры для тестирования.
- Кто отвечает за итоговое поведение программы?

Речь ведущего

- Напомнить кейс: у нас есть требования и общая история кода, теперь хотим ускорить отдельную задачу.
- Нейросеть может помочь с черновиком, объяснением и поиском идей. Уверенный тон не является проверкой правильности.
- Ответственность за принятое в проект решение остаётся у команды.

Вопросы и действия участников

- Быстро спросить, кто уже использовал ИИ для учёбы или кода; 1-2 примера без обсуждения конкретных подписок.

Раскрытие и переход

- Хороший следующий шаг - не просить «сделай весь проект», а выделить небольшую проверяемую задачу.
- Переход к составлению запроса.

Экран 16. Составляем проверяемый запрос

3 мин | 47:00-50:00

Базовая вёрстка: Плохой запрос и пять полей хорошего; затем готовый пример.

На экране

- Слишком расплывчато: «Напиши запись на мероприятия».
- Контекст → задача → ограничения → примеры → проверка.
- Нужно: функция на Python, которая разрешает запись только при наличии места.

Речь ведущего

- Контекст: учебный сервис записи. Задача: вернуть логическое значение, разрешена ли новая запись.
- Ограничения примера: booked и capacity - целые неотрицательные числа; повторные записи пока проверяет другая часть системы.
- Примеры: 9 из 10 - можно; 10 из 10 - нельзя. Попросить объяснить условие и дать тесты.
- Чёткий запрос помогает оценить результат, но не гарантирует, что ответ верен.

Вопросы и действия участников

- Команды предлагают недостающее условие или пример.
- Собрать запрос на экране последовательно, а не сразу показывать весь ответ.

Раскрытие и переход

- Запрос ограничивает задачу до понятной функции.
- Переход: «Вот специально подготовленный ответ в стиле ИИ. Проверим его».

Важно: Ответ на следующем экране - учебная заготовка с намеренной ошибкой, а не результат обязательного живого обращения к сервису.

Экран 17. Убедительный код с одной ошибкой

5 мин | 50:00-55:00

Базовая вёрстка: Короткий код и таблица примеров без ответов.

На экране

- Условия: booked - число занятых мест; capacity - вместимость.
- Вопрос: когда функция разрешит запись лишнего участника?
- Проверим: (9, 10), (10, 10), (0, 0).

```
def can_register(booked, capacity):
    return booked <= capacity
```

Речь ведущего

- Прочитать функцию по строкам; <= означает «меньше или равно».
- Не сообщать ошибку до обсуждения. Попросить сначала предсказать результат для каждого примера.
- Связать поведение с критерием: при заполненном мероприятии новая запись запрещена.

Вопросы и действия участников

- 60 секунд обсуждения; затем представители дают результаты и объясняют один из них.
- Если группа затрудняется, подсказка: «При booked=capacity есть хотя бы одно свободное место?»

Раскрытие и переход

- Функция возвращает True для всех трёх примеров. Для (10, 10) и (0, 0) это неверно по условию.
- Ошибка - разрешено равенство. Переход: исправим условие и закрепим примерами.

Экран 18. Исправляем и проверяем границы

3 мин | 55:00-58:00

Базовая вёрстка: Исправленный код; ожидаемые результаты; ниже одно ограничение.

На экране

- (9, 10) → True; (10, 10) → False; (0, 0) → False.
- Дополнительный пример: (11, 10) → False.
- Один тест «всё работает» не проверяет все границы.

```
def can_register(booked, capacity):
    return booked < capacity
```

Речь ведущего

- Показать замену <= на < и повторить все примеры, а не только тот, на котором нашли ошибку.
- В реальном приложении эта функция не защищает от повторной записи, параллельных запросов и неправильных типов.
- Проверка значения - лишь одна часть общего процесса; не объявлять эту функцию готовой серверной записью.

Вопросы и действия участников

- Спросить: «Какой пример вы бы добавили и что он проверяет?»
- Принять вариант с нулём, полным мероприятием или отсутствием мест.

Раскрытие и переход

- В набор: ИИ для небольшой задачи + понятные ограничения + независимая проверка результата.
- Переход: «А что из проекта допустимо отправлять в запрос?»

Экран 19. Что можно отправить помощнику?

2 мин | 58:00-60:00

Базовая вёрстка: Три примера входных материалов; затем ответ.

На экране

- A. Учебная функция с вымышленными значениями.
- B. Список настоящих школьников с телефонами.
- C. Пароль от сервера для поиска ошибки подключения.

Речь ведущего

- Для демонстрации используем искусственные данные и минимальный фрагмент кода.
- Не передаём пароли и персональные данные просто ради удобства. У сервиса и организации могут быть отдельные правила использования.
- Сначала убираем лишнее и обезличиваем пример; если секрет уже раскрыт, удалить сообщение недостаточно - секрет нужно заменить.

Вопросы и действия участников

- Быстрое голосование за подходящий материал для нашего учебного запроса.

Раскрытие и переход

- A подходит для занятия. B и C в таком виде не отправляем.
- Переход: «Теперь превратим разовую ручную проверку в повторяемый процесс».

Управление временем: Не обсуждать обзоры моделей, тарифы и сравнения брендов: это отдельная тема.

Экран 20. Какие проверки действительно нужны?

3 мин | 60:00-63:00

Базовая вёрстка: Набор ситуаций; после обсуждения - ожидаемое поведение.

На экране

- Свободное место есть / мест нет / последнее место.
- Повторная запись / некорректные данные.
- Не работает кнопка на телефоне.
- Что проверяется функцией, а что - приложением целиком?

Речь ведущего

- Показать разные масштабы: условие функции, взаимодействие частей, путь пользователя в браузере.
- Ожидаемый ответ задаётся требованиями; тест не должен просто копировать написанную реализацию.
- Внешний вид, доступность интерфейса и понятность сообщения тоже требуют проверки, часть из неё остаётся ручной.

Вопросы и действия участников

- Каждая команда называет один тест: исходное условие → действие → ожидаемый результат.
- Пример для помощи: заполненное мероприятие → попытка записи → отказ без добавления участника.

Раскрытие и переход

- Критичный путь записи проверяем раньше второстепенных украшений.
- Переход: «Кто будет выполнять эти проверки после каждого изменения?»

Экран 21. CI: проверяем изменения автоматически

3 мин | 63:00-66:00

Базовая вёрстка: Три этапа проверки и условный результат.

На экране

- Изменение в репозитории → автоматическая проверка → результат для команды.
- Пример результата: 3 проверки прошли, 1 обнаружила ошибку.
- Инструмент-пример: GitHub Actions или GitLab CI/CD.

Речь ведущего

- CI - практика частого объединения изменений с автоматизированными проверками; один запуск тестов ещё не описывает весь процесс.
- Проверки могут запускаться при предложении изменений и при обновлении общей ветки. Конкретный набор команда настраивает.
- Если проверка обязательная, её ошибка может блокировать объединение. Это правило надо отдельно настроить.

Вопросы и действия участников

- Спросить: «Если тест не прошёл, обязательно ли ошибочен код?»
- Принять варианты: ошибка в коде, неверный тест, проблема среды; нужен разбор причины.

Раскрытие и переход

- Красный результат - сигнал для расследования, зелёный - успешное прохождение заданных проверок.
- Зелёный статус не доказывает отсутствие всех ошибок.
- Переход: «Проверка закончилась. Когда пользователи увидят новую версию?»

Экран 22. CD: как изменения доходят до пользователя

3 мин | 66:00-69:00

Базовая вёрстка: Два маршрута после проверок: с подтверждением и без него.

На экране

- Проверки → сборка → тестовое окружение → подтверждение → рабочая среда.
- Continuous delivery: версия подготовлена к выпуску, решение о выпуске может быть ручным.
- Continuous deployment: прошедшие проверки изменения выпускаются автоматически.

Речь ведущего

- Различить проверку кода и размещение новой версии приложения.
- Сборка подготавливает приложение к запуску; её конкретные шаги зависят от проекта.
- CD используют в двух значениях, поэтому важно проговаривать, какая автоматизация имеется в виду.
- Для нашего примера оставляем подтверждение перед демонстрационной версией: это выбранное правило, не универсальный стандарт для всех проектов.

Вопросы и действия участников

- Назвать временного «ответственного за выпуск» и спросить, какие результаты он хочет увидеть перед подтверждением.

Раскрытие и переход

- Понятный порядок выпуска помогает повторять действия и замечать проблемы.
- Переход: «А если уже выпущенное изменение сломало сервис?»

Экран 23. Возврат версии и резервная копия

3 мин | 69:00-72:00

Базовая вёрстка: Две аварии и два разных способа восстановления.

На экране

- Авария 1: новая версия кода ломает запись.
- Авария 2: потеряна база записавшихся участников.
- Предыдущая версия приложения ≠ резервная копия данных.

Речь ведущего

- При неудачном обновлении может помочь возврат на известную рабочую версию.
- Потерянные записи восстанавливаются из резервной копии данных, если она есть и пригодна для восстановления.
- Откат может осложниться изменением структуры данных; это не волшебная кнопка для любого сбоя.
- Не все данные после последней копии удастся вернуть. Важно не только делать копии, но и проверять восстановление.

Вопросы и действия участников

- Попросить сопоставить две аварии с действиями.
- Дополнительный вопрос: «Если копия хранится только на том же сломанном диске, достаточно ли этого?»

Раскрытие и переход

- В набор: проверки + автоматизация + порядок выпуска + план восстановления.
- Переход: «Выберем разумный минимум перед показом».

Экран 24. Выбираем условия выпуска

3 мин | 72:00-75:00

Базовая вёрстка: Три варианта процесса; затем сводка набора.

На экране

- A. Выпускаем всё сразу после сохранения файла.
- B. Проверяем основной путь, смотрим результат review и автоматических проверок, знаем предыдущую версию.
- C. Откладываем любой показ, пока не докажем отсутствие всех ошибок.

Речь ведущего

- Уточнить: демонстрация с тестовыми данными, критичный путь - корректная запись.
- Различить учебный показ и настоящий запуск с реальными пользователями: требования к последнему могут быть шире.
- Попросить оценить не только скорость, но и последствия отказа.

Вопросы и действия участников

- Голосование, один аргумент и одно ограничение выбранного варианта.

Раскрытие и переход

- A: слабый контроль. B: разумный минимум для указанного показа, но не универсальный чек-лист промышленного запуска. C: абсолютное доказательство отсутствия ошибок здесь недостижимо.
- Все пять разделов набора собраны. Дальше - общая ситуация без новых терминов.

Экран 25. До демонстрации осталось два часа

2 мин | 75:00-77:00

Базовая вёрстка: Условие и четыре факта; без подсказки с ответом.

На экране

- Заказчик просит срочно добавить уведомления.
- Проверка записи иногда падает.
- Один разработчик недоступен.
- До демонстрации - два часа.
- Выберите три первых действия и расположите их по порядку.

Речь ведущего

- Напомнить цель показа: продемонстрировать корректную запись, а не максимальное количество функций.
- Уточнить: причина падающей проверки пока неизвестна. Не предполагать, что её можно безопасно отключить.
- Разрешить командам обращаться к собранному набору на следующем экране.

Вопросы и действия участников

- Дать 30 секунд на уточняющие вопросы.
- Не подтверждать «правильность» предложений до общего обсуждения.

Раскрытие и переход

- Переход к экрану действий; каждая команда готовит один общий ответ.

Экран 26. Соберите план спасения демонстрации

4 мин | 77:00-81:00

Базовая вёрстка: Список действий и пустые позиции 1/2/3.

На экране

- A. Уточнить цель и согласовать перенос уведомлений.
- B. Воспроизвести падение проверки и установить причину.
- C. Найти рабочую версию и проверить основной путь.
- D. Отключить падающий тест без выяснения причины.
- E. Попросить ИИ дописать уведомления и сразу выпустить их.
- F. Выяснить доступность материалов и перераспределить работу.

Речь ведущего

- Дать 90 секунд обсуждения; предупредить за 20 секунд до конца.
- У каждой команды три действия с порядком, а не просто три буквы.
- Попросить представителей двух команд объяснить план; остальным - назвать отличие или риск.

Вопросы и действия участников

- Если ответы совпали, спросить: «Какое действие можно выполнить параллельно и кто за него отвечает?»
- Голосование за общий план зала после двух выступлений.

Раскрытие и переход

- Сначала сохранить выбранный залом ответ для разбора.
- Не показывать баллы и проценты успешности: оснований для объективного расчёта здесь нет.

Управление временем: Если времени мало: 60 секунд обсуждения и одно выступление, остальные дополняют поднятием руки.

Экран 27. Разбираем решения и меняем одно условие

4 мин | 81:00-85:00

Базовая вёрстка: План зала; затем пример разумного решения и вопрос «что если».

На экране

- Пример: согласовать объём → понять сбой → проверить стабильный основной путь.
- Параллельно: перераспределить работу по доступной документации и репозиторию.
- Что изменится, если демонстрация проходит через два дня, а не два часа?

Речь ведущего

- A помогает согласовать ожидания. B отделяет дефект от неверной проверки или среды. C даёт безопасную основу показа, если версия действительно проверена.
- F полезно выполнить параллельно: недоступность коллеги не должна означать потерю всех материалов.
- D скрывает сигнал без понимания причины. E добавляет непроверенный объём в момент неопределённости.
- Если причиной оказался неверный тест, его исправляют обоснованно; это отличается от удаления ради зелёного статуса.

Вопросы и действия участников

- Попросить одну команду изменить свой план при новом сроке.
- Спросить, какую небольшую задачу теперь можно поручить ИИ и как проверить ответ.

Раскрытие и переход

- При большем запасе времени можно переоценить объём и добавить проверяемый шаг; обещать новую функцию автоматически не нужно.
- Смысл инструментария - выбирать действия по ситуации, а не всегда применять весь набор.

Экран 28. Пять вопросов перед любым проектом

3 мин | 85:00-88:00

Базовая вёрстка: Пять вопросов и соответствующие части набора, раскрываемые по очереди.

На экране

- Что делаем? → требования, MVP, критерии приёмки.
- Как организуем? → обратная связь, короткие циклы, доска задач.
- Как работаем с кодом вместе? → Git, репозиторий, review.
- Как ускоряемся? → ИИ с контекстом и проверкой.
- Как выпускаем? → тесты, CI/CD, восстановление.

Речь ведущего

- Не читать определения заново. Предлагать назвать решение до раскрытия правой части.
- Отметить: GitHub, доска и нейросеть не исправляют процесс автоматически. Важно, что команда с их помощью делает.
- Можно начать с простого набора и добавлять практики по мере появления реальной потребности.

Вопросы и действия участников

- Попросить разные команды ответить на разные вопросы.
- Если термин забыли, принять правильное объяснение своими словами.

Раскрытие и переход

- Переход: «Назовите один инструмент, который вы теперь сможете применить осознанно».

Экран 29. Выходной вопрос

2 мин | 88:00-90:00

Базовая вёрстка: Одна фраза с пропусками и завершение.

На экране

- Когда в проекте возникает _____, я попробую _____, потому что _____.
- Спасибо за решения и обсуждение.

Речь ведущего

- Выслушать 3-4 коротких ответа из разных команд.
- При необходимости уточнить причинную связь: «Как именно это поможет?»
- Завершить: инструменты не гарантируют срок, но помогают видеть работу, раньше замечать ошибки и принимать обоснованные решения.

Вопросы и действия участников

- Не проводить отдельный тест на запоминание английских слов.
- Если остаётся время, принять один вопрос от зала.

Раскрытие и переход

- МК завершён. Не обещать проведение второго МК в этот же день, пока его расписание не согласовано.

Управление временем: Если времени нет, попросить всех сформулировать ответ соседу; выслушать один пример.

Памятка ведущему перед проведением

Что подготовить

- Открыть сценарий на отдельном устройстве или распечатать. На проектор выводить только материалы для участников.
- Заранее проверить читаемость короткого кода и условий с последнего ряда. Не использовать живую генерацию ответа ИИ как обязательную часть МК.
- Продумать, как фиксировать выбор зала: буква варианта и один главный аргумент. Точные количества голосов необязательны.
- Подготовить два примера для демонстрации проверки: неверный оператор <= и исправленный <. Не менять их случайно местами.
- Для доски задач достаточно заранее подготовленных состояний; не требуется отдельный аккаунт в Trello или GitHub.

Если группа молчит

- Сначала дать 20-30 секунд обсудить с соседом; затем спросить конкретную команду.
- Вместо «Кто знает Git?» спросить «Как вы поймёте, какой файл последний?»
- Предложить два конкретных примера и попросить найти отличие. Принять ответ обычными словами, потом назвать термин.

Если обсуждение затянулось

- Сокращать число выступлений, а не убирать объяснение последствий. Один ответ + одно дополнение обычно достаточны.
- Сократить выходы к доске на экранах 08 и 12; перемещать элементы по предложениям зала самому.
- Не расширять разговор в настройки Git, установку сервисов, тарифы ИИ и синтаксис CI-конфигурации.
- Сохранить разбор ошибки Python, финальную ситуацию и итоговые пять вопросов. Это опоры понимания.

Если осталось время

- Вернуться к одному выбору: изменить срок, доступность коллеги или критичность ошибки и попросить пересмотреть решение.
- Дополнительный пример: два человека одновременно видят последнее место. Одна проверка < в Python не предотвращает ситуацию, когда оба успевают записаться.
- Объяснить без кода: проверку и фиксацию записи нужно согласованно выполнять на стороне системы хранения, а не полагаться только на кнопку в браузере.

Короткие ответы и границы упрощений

Agile и Scrum - одно и то же?

Нет. Agile описывает ценности и принципы. Scrum задаёт конкретный фреймворк работы. Доску задач и короткие циклы можно использовать и вне Scrum.

MVP - это некачественный продукт?

Нет. Ограничивается объём, а не необходимость корректно выполнять выбранную основную задачу. В нашем случае запись не должна превышать лимит мест.

Критерии приёмки и Definition of Done?

Критерии приёмки уточняют ожидаемое поведение конкретного требования. Definition of Done в Scrum задаёт общее понимание качества готового инкремента. На занятии достаточно первого понятия; второе вводить только по вопросу.

Коммит уже доступен всей команде?

Не обязательно. Он может оставаться только в локальном репозитории. Изменения ещё нужно передать в общий репозиторий.

GitHub - это Git?

Нет. Git - система контроля версий. GitHub - платформа, использующая Git и добавляющая совместные инструменты. Существуют другие платформы.

Review всегда должен делать мидл?

Нет. Нужен человек, способный оценить изменение. В учебной команде можно взаимно проверять небольшие понятные правки.

Зелёные тесты означают, что багов нет?

Нет. Пройдены только заданные проверки в заданных условиях. Могут отсутствовать важные сценарии или оказаться неверными сами ожидания.

CI/CD - обязательно автоматический выпуск?

Нет. Можно автоматически проверять и готовить версию, но оставлять подтверждение выпуска человеку. Continuous deployment означает автоматический выпуск после прохождения заданных проверок.

Git заменяет резервную копию сервера?

Нет. История исходного кода не равна резервной копии рабочих данных, конфигурации и всего окружения.

Справочные материалы и рамки сценария

Официальные материалы нужны ведущему для уточнения терминов. Открывать их во время занятия необязательно. Проверено 27 октября 2026 года.

Scrum Guide 2020

https://scrumguides.org/scrum-guide.html

Определение Scrum, роли ответственности, события и артефакты. В сценарии они представлены выборочно и упрощённо; МК не воспроизводит полный Scrum-процесс.

Pro Git: About Version Control

https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control

Введение в контроль версий. Подходит для последующего самостоятельного знакомства с Git.

GitHub Docs: Understand GitHub Actions

https://docs.github.com/en/actions/get-started/understand-github-actions

Пример платформы для автоматизации проверок, сборки и размещения приложения. Настройка workflow не входит в содержание МК.

Что здесь является учебной моделью

- Сервис фестиваля, срок в одну неделю, сбои и варианты действий придуманы для обсуждения. Они не задают оценку стоимости или сроков настоящего проекта.
- Буквы вариантов - опора для голосования, а не тест с универсально верными ответами. Ведущий уточняет условия и обсуждает ограничения.
- Код Python проверяет только числовой лимит при оговорённых входных данных. Он не является реализацией регистрации.
- Числовой рейтинг готовности и вероятности успеха для первого МК не рассчитываются. Его результат - объяснимые решения и собранный набор практик.

Следующий этап подготовки

- После согласования сценария: спроектировать структуру общего сервиса на FastAPI + React + TypeScript, формат сценария, состояния экранов и управление ведущего.
- Первую версию предусмотреть для ручного голосования. Позже можно добавить комнаты, телефоны и QR-коды без изменения содержания этих экранов.
- Сценарий второго МК разрабатывается отдельно; понятия из этого занятия станут его исходным инструментарием.